

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA1214	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	M.Koushik Reddy	Reg. No.:	192372185
Section / Batch:	CSE-AI	Date:	25-08-2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time monitoring system receives data from multiple sensors with different sampling frequencies and response-time requirements. Develop a pseudocode for an interrupt scheduling mechanism that assigns suitable interrupt priorities to the sensors based on their sampling rate, urgency, and data-processing requirements. The algorithm should manage simultaneous interrupt requests and ensure that time-critical sensor data is serviced without causing excessive CPU overhead.
2. A high-speed data acquisition system must transfer continuous streams of data between an I/O device and memory while preventing data loss when the incoming data rate exceeds the processing speed of the CPU. Write a pseudocode for a double-buffered DMA mechanism that alternates between two memory buffers during data transfer. The algorithm should detect buffer completion, switch buffers, generate appropriate interrupts, and handle buffer overflow conditions.
3. A computer system contains several I/O devices with different data transfer sizes and timing requirements. Design a pseudocode for an I/O scheduling algorithm that determines the order in which pending I/O requests should be serviced. The algorithm should consider device priority, request size, waiting time, latency requirements, and available bandwidth to improve overall I/O performance while preventing starvation.
4. A high-performance embedded system uses multiple DMA-capable devices that may request memory access at the same time. Develop a pseudocode for a DMA channel arbitration mechanism that allocates memory bus access among competing DMA requests. The algorithm should consider request priority, transfer size, urgency, and fairness, while ensuring that high-priority transfers are completed within their required time constraints.

5. A real-time system must communicate simultaneously with several devices that have different communication speeds and reliability requirements. Write a pseudocode for an adaptive I/O management mechanism that monitors device performance and dynamically changes the communication method or transfer parameters when congestion, delays, or errors are detected. The algorithm should monitor transfer status, detect performance degradation, adjust resource allocation, and maintain reliable communication with minimal CPU overhead.

Code of Conduct:-

I M.Koushik Reddy/192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Koushik Reddy

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator Faculty Incharge:_____

Objective

Assign interrupt priorities to sensors based on sampling rate, urgency, and processing load; handle simultaneous interrupt requests; and keep CPU overhead bounded so time-critical data is never missed.

Pseudocode

```
BEGIN

  Initialize interrupt queue
  Initialize CPU and sensor parameters

  FOR each sensor DO
    Calculate Priority based on:
      Sampling Rate
      Urgency
      Data Processing Requirement

    Assign interrupt priority
    Add sensor to interrupt queue
  END FOR

  WHILE system is running DO

    IF interrupt request is received THEN
      Add request to interrupt queue
      Sort queue according to priority
    END IF

    IF CPU is free AND queue is not empty THEN
      Select highest-priority interrupt

      IF interrupt is time-critical THEN
        Service interrupt immediately
      ELSE
        Service interrupt according to priority
      END IF

      Process sensor data
      Clear interrupt flag
    END IF

    IF multiple interrupts occur simultaneously THEN
      Select request with:
        Highest urgency
        Higher sampling rate
        Higher processing requirement
    END IF

    Reduce unnecessary context switching
```

```
Return CPU to normal operation
```

```
END WHILE
```

```
END
```

Explanation

Each sensor is registered with a sampling rate, an urgency rating, and a processing load. A weighted score combining normalized sampling rate, urgency, and deadline tightness maps every sensor onto a discrete interrupt priority level, so the fastest and most time-critical sensors occupy the highest vector. When multiple interrupts arrive together, a priority queue orders them and the scheduler always services the highest-priority pending request first, while temporarily masking lower-priority lines to avoid priority inversion. To keep CPU overhead bounded, heavy processing is deferred out of the interrupt context, bursty low-priority sensors are coalesced, and an aging routine gradually raises the priority of long-waiting requests so no sensor is starved indefinitely.

Key design points

Weighted priority scoring • priority-queue dispatch • priority-inversion avoidance via masking • deferred processing to shorten ISR time • interrupt coalescing for bursty sensors • aging to prevent starvation.

Objective

Transfer continuous high-speed data between an I/O device and memory without loss, by alternating between two buffers, detecting buffer completion, switching buffers, generating interrupts, and handling overflow.

Pseudocode

```
BEGIN

    Create Buffer_A
    Create Buffer_B

    Set Active_Buffer = Buffer_A
    Set Processing_Buffer = Buffer_B

    Initialize DMA Controller
    Enable DMA Interrupt

    START DMA transfer into Active_Buffer

    WHILE data acquisition is running DO

        IF DMA transfer is complete THEN

            Generate DMA completion interrupt

            Mark Active_Buffer as FULL

            Swap buffers:
                TEMP = Active_Buffer
                Active_Buffer = Processing_Buffer
                Processing_Buffer = TEMP

            START DMA transfer into Active_Buffer

            Process Processing_Buffer

            IF Processing_Buffer is not processed
                before next DMA completion THEN
                Detect Buffer Overflow
                Raise Overflow Warning
                Increase processing priority
                OR temporarily reduce input rate
            END IF

        END IF

    END IF

    IF DMA reports transfer error THEN
        Stop DMA
        Report error
    END IF
```

```

Restart DMA transfer
END IF

END WHILE

END

```

Explanation

Two equally sized buffers, A and B, alternate roles between "active" (currently receiving data from the device via DMA) and "drain" (currently being consumed by the CPU). Every incoming data unit advances an index into the active buffer; once that index reaches the buffer's capacity, a buffer-full interrupt fires. The handler checks whether the CPU has finished reading the other buffer — if so, the two buffers instantly swap roles so the DMA engine never stalls, and the just-filled buffer is handed to a CPU task for processing. If the CPU has not kept up, an overflow condition is raised instead, and a mitigation policy (discarding, compressing, growing the buffer, or throttling the source) is applied. This keeps data capture continuous even though CPU consumption happens in bursts rather than in lock-step with the incoming stream.

Diagram — Double-Buffering Data Flow

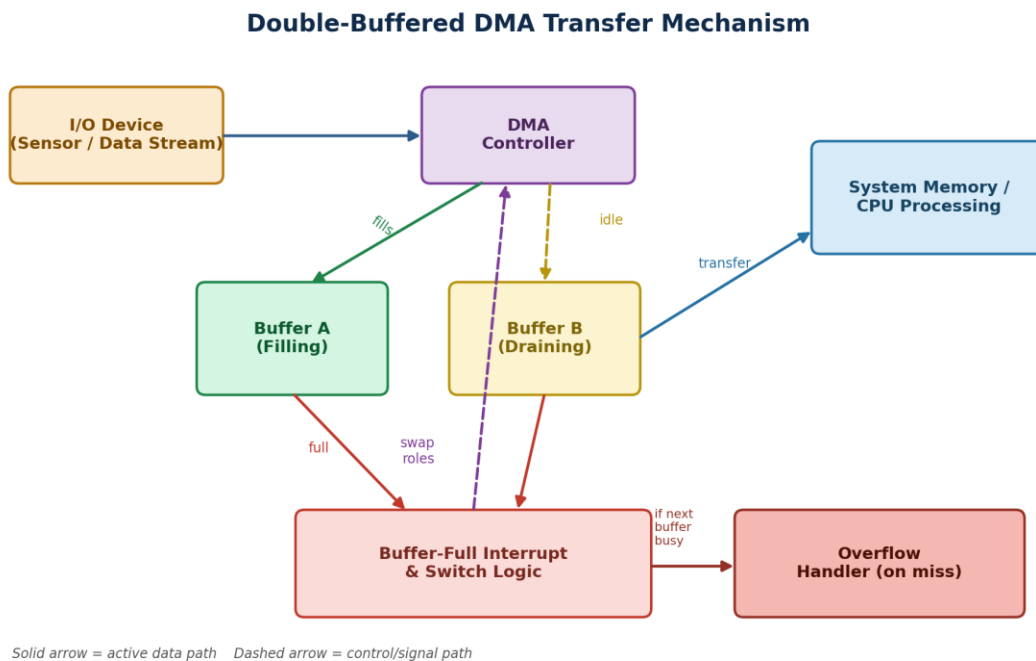


Figure 1: While Buffer A fills from the device, Buffer B drains into system memory; on buffer-full the interrupt/switch logic swaps their roles, or raises an overflow condition if the CPU has not finished draining.

Objective

Order pending I/O requests from devices with different sizes and timing needs, considering device priority, request size, waiting time, latency requirements, and available bandwidth, while preventing starvation.

Pseudocode

```
BEGIN

  Initialize Pending_IO_Queue

  WHILE I/O requests are available DO

    FOR each pending I/O request DO

      Calculate Scheduling Score using:
        Device Priority
        Request Size
        Waiting Time
        Latency Requirement
        Available Bandwidth

      Increase priority if waiting time is high

      IF latency requirement is critical THEN
        Increase scheduling score
      END IF

    END FOR

    Select request with highest Scheduling Score

    IF selected device is ready THEN
      Start I/O transfer
    ELSE
      Select next suitable request
    END IF

    Update available bandwidth

    WHILE I/O transfer is running DO
      Monitor transfer status

      IF transfer is completed THEN
        Generate completion interrupt
        Remove request from queue
      END IF
    END WHILE

    Increase waiting time of remaining requests
```

```
    IF any request waits for too long THEN
        Increase its priority
    END IF

END WHILE

END
```

Explanation

Every pending I/O request carries a base priority, a size, an arrival time, a maximum tolerable latency, and a bandwidth requirement. A composite score blends priority, an urgency term derived from the latency bound, an aging term based on wait time, and a size term that discourages a single large transfer from dominating the channel. Requests are resorted by this score each cycle, the top request is serviced if enough bandwidth is currently available (otherwise it is requeued with a short backoff), and the aging term is boosted for everything left behind — which guarantees that even low-priority, low-bandwidth requests eventually rise to the top and avoid starvation.

Key design points

Composite scoring (priority + urgency + wait + size) • bandwidth-aware dispatch with requeue-on-shortage • continuous aging so no request waits forever • SLA-violation logging for latency accountability.

Objective

Allocate shared memory-bus access among multiple simultaneous DMA requests, considering priority, transfer size, urgency, and fairness, while guaranteeing high-priority transfers meet their time constraints.

Pseudocode

```
BEGIN

  Initialize DMA request queue

  WHILE system is running DO

    Detect all DMA requests

    FOR each DMA request DO
      Calculate Arbitration Score using:
        Request Priority
        Transfer Size
        Urgency
        Waiting Time
    END FOR

    IF any request is time-critical THEN
      Select the most urgent time-critical request
    ELSE
      Select request with highest Arbitration Score
    END IF

    IF two requests have equal priority THEN
      Select the request that has waited longer
    END IF

    Grant memory bus access to selected DMA channel

    Transfer required data

    IF transfer is completed THEN
      Release memory bus
      Generate DMA completion interrupt
      Remove request from queue
    END IF

    Update waiting time of remaining requests

    IF a low-priority request waits for too long THEN
      Increase its priority
    END IF

  END WHILE
```

END

Explanation

When several DMA-capable devices assert bus-request lines simultaneously, each pending request is assigned an arbitration weight built from its priority, an urgency term (how close its deadline is), a fairness/aging term (how long it has waited), and a penalty term for large transfer sizes so bulky transfers don't monopolize the shared bus. The request with the highest weight is granted the bus; a hard real-time guard allows an urgent request to pre-empt a lower-priority transfer already in progress if its deadline would otherwise be missed. As soon as a transfer completes, an interrupt re-triggers arbitration for the next winner, and an aging step periodically nudges up the priority of requests that have waited the longest.

Key design points

Weighted arbitration score • deadline-driven pre-emption for hard real-time transfers • immediate re-arbitration on completion interrupt • aging increment for long-waiting channels to guarantee fairness.

Objective

Monitor multiple devices with different speeds and reliability, and dynamically adjust the communication method or transfer parameters when congestion, delay, or errors are detected — while keeping CPU overhead minimal.

Pseudocode

```
BEGIN

  Initialize all I/O devices
  Initialize communication parameters

  WHILE system is running DO

    FOR each device DO

      Monitor:
        Transfer Rate
        Response Time
        Error Rate
        Queue Length
        Communication Delay

      IF congestion is detected THEN
        Reduce transfer size
        Allocate additional bandwidth
        Increase communication priority
      END IF

      IF response delay increases THEN
        Adjust timeout value
        Increase device priority
        Select faster transfer method if available
      END IF

      IF error rate increases THEN
        Enable error detection
        Retry failed transfer

        IF errors continue THEN
          Reduce transfer speed
          Reduce packet/block size
          Reinitialize communication
        END IF
      END IF

      IF device performance improves THEN
        Gradually restore normal parameters
        Release unnecessary resources
      END IF

    END IF

  END WHILE
```

```
        Update device performance statistics

    END FOR

    Generate interrupt only when:
        Transfer completes
        Error occurs
        Performance threshold is exceeded

    Use polling only when necessary

END WHILE

END
```

Explanation

A background monitor periodically samples each active communication link's error rate, throughput, and latency. If a link's error rate or latency crosses a threshold, the system first retries the transfer a bounded number of times (to absorb transient noise); if degradation persists, it steps the link down through progressively more conservative communication modes — reduced speed, then stronger error correction, then failover to a backup channel entirely — and logs the adaptation. When a degraded link's throughput recovers, the system cautiously steps it back up one mode at a time, re-checking stability before trusting the faster mode again. All monitoring and mode switching runs as lightweight background/deferred tasks rather than inside interrupt context, keeping CPU overhead low while maintaining reliable communication across devices with very different speed and reliability profiles.

Key design points

Rolling performance sampling • bounded retries before mode change • staged degradation (reduced speed → error correction → failover) • cautious step-wise recovery • all monitoring runs as background/deferred tasks to minimize CPU overhead.